

Sorting

1. Elementary Sorting

1.1 Selection Sort Algorithm

基本思想：从输入中选出最小的元素，然后递归地对剩下的元素排序，最后把最小的元素和已排好序的元素合并

```
1 SelectionSortRec(A):
2   if A.length=1
3       return A
4   else
5       min:=GetMinElement(A)
6       A*=RemoveElement(A,min)
7       return Concatenate(min,SelectionSortRec(A*))
8
9 SelectionSort(A):
10  for i:=1 to A.length
11      minIdx:=i
12      for j:=i+1 to A.length
13          if A[j]<A[minIdx]
14              minIdx:=j
15      Swap(i,minIdx)
```

- 运行时间为 $\Theta(n^2)$
- 空间复杂度为 $\Theta(1)$
- 不稳定，交换操作会扰乱相关的顺序

1.2 Bubble Sort Algorithm

基本思想：重复遍历数组，比较相邻的元素对并且交换，这样更大的元素就“冒泡升到”数组的顶部

```
1 BubbleSort(A):
2   for i:=A.length down to 2
3       for j:=1 to i-1
4           if A[j]>A[j+1]
5               Swap(A[j],A[j+1])
```

- 时间复杂度为 $\Theta(n^2)$
- 空间复杂度为 $\Theta(1)$
- 稳定

改进版BubbleSort:

```

1 BubbleSortImproved(A):
2   n:=A.length
3   repeat
4     swapped:=false
5     for j:=1 to n-1
6       if A[j]>A[j+1]
7         Swap(A[j],A[j+1])
8         swapped:=true
9     n:=n-1
10  until swapped=false

```

特别地，当输入已经为有序的时，只须 $O(n)$ 时间。最差的情况也需要 $\Theta(n^2)$ 时间。

1.3 Insertion Sort Algorithm

```

1 InsertSort(A):
2   for i:=2 to A.length
3     key:=A[i]
4     j:=i-1
5     while j>0 and A[j]>key
6       A[j+1]:=A[j]
7       j:=j-1
8     A[j+1]:=key
9   return A

```

- 改进版InsertionSort:
- 当输入的数组容量很小且将近有序，InsertionSort是高效的。
- 允许元素可以移动很大的步长。

1.4 Comparison of simple sorting algorithms

1. Insertion

- $n(n-1)/2$ swaps, and $n(n-1)/2$ comparisons $\rightarrow$ worst
- $n(n-1)/4$ swaps, and $n(n-1)/4$ comparisons $\rightarrow$ on average

2. Selection

- $n-1$ swaps, and $n(n-1)/2$ comparisons

3. Bubble

- $n(n-1)/2$ swaps, and $n(n-1)/2$ comparisons

1.5 ShellSort

To sort n items, define a set of decreasing distances $\{d_1, d_2, \dots, d_k\}$ with $d_1 < n$ and $d_k = 1$.

ShellSort then go through k passes, for the i^{th} pass:

- Divide items into d_i groups each of size about n/d_i , and the j^{th} group contains items with index $j, j+d_i, j+2d_i, j+3d_i, \dots$
- For each of the d_i groups, sort the items in that group. (uses **InsertionSort**)

```

1  #include <vector>
2  #include<iostream>
3
4  using namespace std;
5
6  void ShellSort(vector<int>& nums) {
7
8      int length=nums.size();
9      int h=1;
10     while(h<length/3){
11         h=h*3+1;
12     }
13     while(h>=1){
14         for(int i=h;i<length;i++){
15             for(int j=i;j>=h&&nums[j]<nums[j-h];j-=h){
16                 int temp=nums[j];
17                 nums[j]=nums[j-h];
18                 nums[j-h]=temp;
19             }
20         }
21         h=h/3;
22     }
23 }

```

ShellSort can be slow!

2.QuickSort

2.1 众多排序算法的一个统一观点

把问题分成若干个子问题，递归地对子问题进行分治，把子问题的解答结合起来。

2.2 The QuickSort Algorithm

- 基本思想：
- 有一个含有 n 个元素的数组 A
- 选择 A 中的一个元素 x 作为 $pivot$
- 用这个 $pivot$ 把输入分成 B 和 C ，这样 B 中的元素均小于等于 x , C 中的元素均大于 x
- 对 B 和 C 进行递归排序
- 输出 $\langle B, x, C \rangle$

2.3 The Partition Procedure

```
1 Partition(A):
2   x:=A[n], l:=1, r:=n
3   for i:=1 to n-1
4       if A[i] <= x
5           B[l]:=A[i]
6           l++
7       else
8           B[r]:=A[i]
9           r--
10  B[l]:=x
11  return <B, l>
```

- 分配大小为 n 的数组 B
- 遍历 A ,把小的元素放在 B 的左边, 大的元素放在 B 的右边
- 最后把 $pivot$ 放在剩下的位置
- $\Theta(n)$ 时间, $\Theta(n)$ 空间, 不稳定

2.4 In-place Partition Procedure

- 遍历数组 A , 使用交换操作把小的元素移动到 A 的左边, 这样 A 的右边自然就剩下大的元素

```
1 InplacePartition(A, p, r):
2   i:=p-1
3   for j:=p to r-1
4       if A[j] <= A[r]
5           i:=i+1
6       swap(A[i], A[j])
7   swap(A[i+1], A[r])
8   return i+1
```

- $\Theta(|r - p|)$ 时间, $O(1)$ 空间, 不稳定

2.5 QuickSort Algorithm

```
1 QuickSort(A, p, r):
2   if p < r
3       q:=InplacePartition(A, p, r)
4       QuickSort(A, p, q-1)
5       QuickSort(A, q+1, r)
```

- 最差的情况: $O(n^2)$ 时间
- balanced: $O(n \log n)$ 时间

2.6 Randomized QuickSort

选择好的 $pivot$ 是很重要的。由于对手论证, 我们考虑随机选择。

```

1 RandQuickSort(A, p, r):
2   if p < r
3     i := Random(p, r)
4     Swap(A[r], A[i])
5     q := InplacePartition(A, p, r)
6     RandQuickSort(A, p, q-1)
7     RandQuickSort(A, q+1, r)

```

- $O(n \log n)$ 时间 maybe

2.7 A bit more on QuickSort

1. 如果有许多相同的元素怎么办？使用“区域”。
2. 对于容量较小的数组，QuickSort是比较慢的。
3. Random pivot selection and Median of three 可以结合使用。
4. 多个 *pivot*?

2.8 The $n \lg n$ sorting algorithms

QuickSort, MergeSort and HeapSort are all with $O(n \lg n)$, which is better?

- HeapSort is non-recursive, minimal auxiliary storage requirement (good for embedded system), but with poor locality of reference, the access of elements is not linear, resulting many caches being missed! It is the slowest among three algorithms.
- In most (not all) tests, Quicksort turns out to be faster than Mergesort. This is because although Quicksort performs 39% more comparisons than Mergesort, but much less movement (copies) of array elements.
- MergeSort is a stable sorting, and can take advantage of partially pre-sorted input. Further, MergeSort is more efficient at handling slow-to-access sequential media.

3. External Sorting

3.1 Definition

外部排序用在当需要被排序的数据无法进入主存设备中时，反之他们必须存储在外部存储中，比如磁盘。

因为 I/O 时间花费很大，所以算法的关键就在减少 I/Os 。

3.2 External merge problem

- 输入：两个排好序的 list，有 M 和 N 页
- 输出：一个归并的排好序的 list，有 $M + N$ 页
- 我们可以用一个大小至少为3的缓冲区来有效归并两个链表吗？
- 是的！并且只要 $2(M + N) I/Os$ ！

关键思想：To find an element that is no larger than all elements in two lists, one only needs to compare minimum elements from each list. Each time put the current minimum elements back to disk.

3.3 2-Way Sort

如何应对没有排序的大文件？

- 分成足够小的块在内存 (runs) 中排序
- 使用外部排序算法对 (runs) 进行归并
- 不断归并得到的 runs，直到得到一个排好序的文件